Computational Data Analysis (02582)

Artificial Neural Networks

Sneha Das

Assistant Professor

Department of Applied Mathematics and Computer Science,

Technical University of Denmark (DTU Compute)

(*Slides adapted: Raghavendra Selvan, DIKU, University of Copenhagen*)

Outline I

- Introduction
- The Ingredients of ANN
- Optimization algorithm - Gradient Descent
- Optimization objective function
- Deep Learning
- Architectures

The Ingredients of Artificial Neural Networks

The Ingredients of Artificial Neural Networks

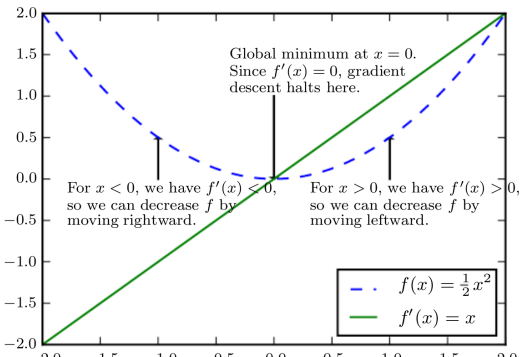
To build and train a modern Deep Learning model, we need four primary ingredients:

1. **Data (D):** Observed features $\mathbf{X}$ and targets $\mathbf{Y}$.
2. **The Objective (L):** A loss function to minimize, usually derived from the **Negative Log-Likelihood**.
3. **The Engine (∇):** An optimizer (Gradient Descent) and an algorithm to find gradients (**Backpropagation**).
4. **Architecture (f_θ):** The choice of non-linear function approximator (MLPs, CNNs, Transformers).

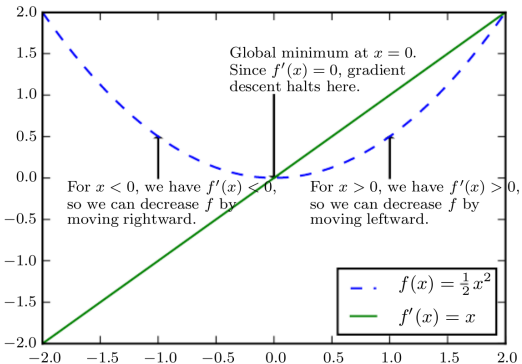
Overview

- Introduction
- The Ingredients of ANN
- Optimization algorithm - Gradient Descent
- Optimization objective function
- Deep Learning
- Architectures

Going downhill or Gradient Descent



Going downhill or Gradient Descent



- **Gradient Descent** is an optimization algorithm.
- Used to minimize a function $L(\mathbf{w})$ by iteratively moving in the direction of steepest descent.

Gradient Descent Algorithm

1. **Initialize:** Choose an initial guess for the parameters $\mathbf{w}$.

Gradient Descent Algorithm

1. **Initialize:** Choose an initial guess for the parameters $\mathbf{w}$.
2. **Compute Gradient:** Calculate the gradient of the objective function $L(\mathbf{w})$ with respect to the parameters, denoted as $\nabla L(\mathbf{w})$.

Gradient Descent Algorithm

1. **Initialize:** Choose an initial guess for the parameters $\mathbf{w}$.
2. **Compute Gradient:** Calculate the gradient of the objective function $L(\mathbf{w})$ with respect to the parameters, denoted as $\nabla L(\mathbf{w})$.
3. **Update Parameters:** Update the parameters in the direction opposite to the gradient:

$$\mathbf{w} = \mathbf{w} - \eta \cdot \nabla L(\mathbf{w})$$

where η is the learning rate, controlling the size of the steps.

Gradient Descent Algorithm

1. **Initialize:** Choose an initial guess for the parameters $\mathbf{w}$.
2. **Compute Gradient:** Calculate the gradient of the objective function $L(\mathbf{w})$ with respect to the parameters, denoted as $\nabla L(\mathbf{w})$.
3. **Update Parameters:** Update the parameters in the direction opposite to the gradient:

$$\mathbf{w} = \mathbf{w} - \eta \cdot \nabla L(\mathbf{w})$$

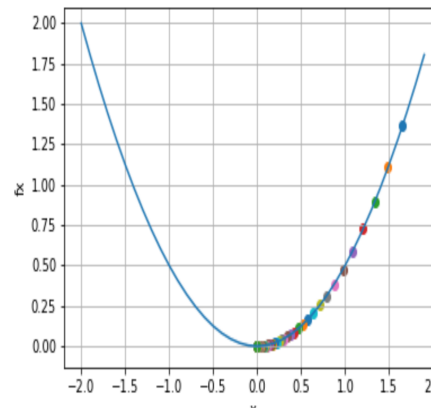
   where η is the learning rate, controlling the size of the steps.
4. **Repeat:** Repeat steps 2 and 3 until convergence or for a specified number of iterations.

Gradient Descent Algorithm

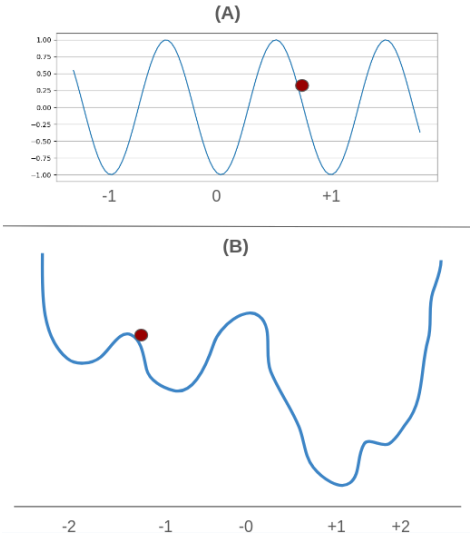
1. **Initialize:** Choose an initial guess for the parameters $\mathbf{w}$.
2. **Compute Gradient:** Calculate the gradient of the objective function $L(\mathbf{w})$ with respect to the parameters, denoted as $\nabla L(\mathbf{w})$.
3. **Update Parameters:** Update the parameters in the direction opposite to the gradient:

$$\mathbf{w} = \mathbf{w} - \eta \cdot \nabla L(\mathbf{w})$$

   where η is the learning rate, controlling the size of the steps.
4. **Repeat:** Repeat steps 2 and 3 until convergence or for a specified number of iterations.
5. **Convergence Criteria:** Stop when the change in the parameters becomes sufficiently small or after a fixed number of iterations.



More on Gradient Descent



Discuss in groups and answer the following questions

A.1 Where does the function attain its minimum?

A.2 If we initialize gradient descent at the red point, where would the model converge?

B.1 Where is the function at its minimum?

B.2 If we initialize gradient descent at red point, where would the model converge?

B.3 How to get the model to converge to another/better minimum?

Activity: The Learning Rate

Minimize $f(w) = w^2$ where $\nabla f(w) = ?$. Start at $w_0 = 10$.

Scenario A: $\eta = 0.1$

$w_1 = ?$

Scenario B: $\eta = 1.1$

$w_1 = ?$

Discussion:

- Which scenario converged?
- What happened in Scenario B? Did we go 'downhill'?

Overview

- Introduction
- The Ingredients of ANN
- Optimization algorithm - Gradient Descent
- Optimization objective function
- Deep Learning
- Architectures

Bridging the Gap: The Engine and the Map

Gradient Descent: as a way to move downhill (minimum $\rightarrow$ loss), but what defines the *terrain*?

Bridging the Gap: The Engine and the Map

Gradient Descent: as a way to move downhill (minimum $\rightarrow$ loss), but what defines the *terrain*?

The Engine: Gradient Descent

Role: The Optimization Algorithm.

- It is the *how*.
- Iteratively updates weights.
- Move opposite to $\nabla_{\mathbf{w}} L$.

The Map: objective function

Role: The Optimization Goal.

- Formulated as a Loss Function $L(\mathbf{w})$.

The Synthesis

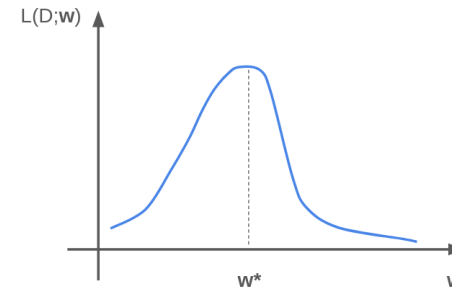
In Neural Networks, we navigate using GD across a terrain defined by the Negative Log-Likelihood (NLL):

$$\text{Minimize } L(\mathbf{w}) \iff \text{Minimize } -\ln \ell(D; \mathbf{w})$$

What are the most common loss functions?

Likelihood function as an optimization objective

- Consider a dataset $D = (\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)$
- The hypothesis is approximating, $h_{\mathbf{w}}() : \mathbf{X} \rightarrow \mathbf{Y}$
- Likelihood function, $\ell(D; \mathbf{w})$ captures how *likely* is it that we get Y from $\mathbf{X}$ if $P(\mathbf{Y}|\mathbf{X})$ was captured by $h_{\mathbf{w}}(\mathbf{x})$
- Due to i.i.d assumptions on the data, $\ell(D; \mathbf{w}) = \prod_{i=1}^{N} \ell((\mathbf{x}_i, y_i); \mathbf{w})$
- Maximum likelihood estimate (MLE) can be obtained by solving for the model parameters that maximize the likelihood function $\mathbf{w}^* = \arg\max_{\mathbf{w}} \ell(D; \mathbf{w})$
- As we like minimizing loss functions in ML, this is equivalent to $\mathbf{w}^* = \arg\min_{\mathbf{w}} -log\ell(D; \mathbf{w})$
- Minimizing the negative log likelihood is how almost all of Deep Learning models are optimized!



Deriving MSE from Gaussian Likelihood

Assume our model output is the mean of a Gaussian distribution: $y_i = h_{\mathbf{w}}(\mathbf{x}_i) + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma^2)$.

- The likelihood for one point is: $p(y_i|\mathbf{x}_i, \mathbf{w}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - h_{\mathbf{w}}(\mathbf{x}_i))^2}{2\sigma^2}\right)$
- The Negative Log-Likelihood (NLL) for the dataset is:

$$-\ln \mathcal{L}(\mathbf{w}) = -\sum_{i=1}^{N} \ln\left[\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - h_{\mathbf{w}}(\mathbf{x}_i))^2}{2\sigma^2}\right)\right]$$

- Expanding the log:

$$-\ln \mathcal{L}(\mathbf{w}) = \underbrace{N \ln(\sqrt{2\pi\sigma^2})}_{\text{constant}} + \frac{1}{2\sigma^2} \sum_{i=1}^{N} (y_i - h_{\mathbf{w}}(\mathbf{x}_i))^2$$

Result

Minimizing NLL under a Gaussian assumption is equivalent to minimizing MSE.

Binary Cross-entropy loss

For classification tasks with binary labels $y_i \in \{0, 1\}$, we require an objective function that measures the distance between our prediction $\hat{y}_i$ and the ground truth.

The BCE Objective

The Binary Cross-entropy (BCE) loss is defined as:

$$L(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^{N} [y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)]$$

where $\hat{y}_i = \sigma(\mathbf{w}^T \mathbf{x}_i)$.

- **Global Minimum:** BCE is convex with respect to the weights in a single-layer network (Logistic Regression).

Activity: Can we derive this formula using Maximum Likelihood Estimation?

Deriving Cross-Entropy from Bernoulli Likelihood

Assume binary classification $y_i \in \{0, 1\}$. Our model output $\hat{y}_i = h_{\mathbf{w}}(\mathbf{x}_i)$ represents $P(y_i = 1|\mathbf{x}_i)$.

- The Bernoulli likelihood for one point is: ?!
- The NLL for the entire dataset:

$$-\ln \mathcal{L}(\mathbf{w}) = ?!$$

- Using log properties:

$$-\ln \mathcal{L}(\mathbf{w}) = ?!$$

Deriving Cross-Entropy from Bernoulli Likelihood

Assume binary classification $y_i \in \{0, 1\}$. Our model output $\hat{y}_i = h_{\mathbf{w}}(\mathbf{x}_i)$ represents $P(y_i = 1|\mathbf{x}_i)$.

- The Bernoulli likelihood for one point is: $p(y_i|\mathbf{x}_i, \mathbf{w}) = \hat{y}_i^{y_i}(1 - \hat{y}_i)^{1-y_i}$
- The NLL for the entire dataset:

$$-\ln \mathcal{L}(\mathbf{w}) = -\sum_{i=1}^{N} \ln \left[\hat{y}_i^{y_i}(1 - \hat{y}_i)^{1-y_i}\right]$$

- Using log properties:

$$-\ln \mathcal{L}(\mathbf{w}) = -\sum_{i=1}^{N} \left[y_i \ln(\hat{y}_i) + (1 - y_i)\ln(1 - \hat{y}_i)\right]$$

Result

This is exactly the Binary Cross-Entropy loss. Minimizing it maximizes the probability of the correct class.

Basic Results from Calculus: Differentiation

- **Definition of Derivative:** Let $f(x)$ be a function. The derivative of $f(x)$ at a point $x = a$, denoted $f'(a)$, is defined as:

$$f'(a) = \lim_{h \to 0} \frac{f(a+h) - f(a)}{h}$$

- **Sum Rule:** If $f(x) = g(x) + h(x)$, then

$$f'(x) = g'(x) + h'(x)$$

- **Power Rule:** If $f(x) = x^n$, where n is a constant, then

$$f'(x) = nx^{n-1}$$

- **Product Rule:** If $f(x) = g(x) \cdot h(x)$, then

$$f'(x) = g'(x) \cdot h(x) + g(x) \cdot h'(x)$$

- **Quotient Rule:** If $f(x) = \frac{g(x)}{h(x)}$, then

$$f'(x) = \frac{g'(x) \cdot h(x) - g(x) \cdot h'(x)}{(h(x))^2}$$

- **Chain Rule:** If $f(x) = g(h(x))$, then

$$f'(x) = g'(h(x)) \cdot h'(x)$$

Common Results: Derivatives of Standard Functions

- **Constant Function:** If $f(x) = c$, where c is a constant, then

$$f'(x) = 0$$

- **First order function of x:** If $f(x) = cx$, where c is a constant, then

$$f'(x) = c$$

- **Exponential Function:** If $f(x) = e^x$, then

$$f'(x) = e^x$$

- **Logarithmic Function:** If $f(x) = \ln(x)$, then

$$f'(x) = \frac{1}{x}$$

- **Fractional Function:** If $f(x) = c/x$, then

$$f'(x) = -\frac{c}{x^2}$$

- **Derivative of Sigmoid function:** $\sigma(x) = 1/(1 - e^{-x})$

$$\frac{d}{dx}(\sigma(x)) = \sigma(x)\,(1 - \sigma(x))$$

Overview

- Introduction
- The Ingredients of ANN
- Optimization algorithm - Gradient Descent
- Optimization objective function
- Deep Learning
- Architectures

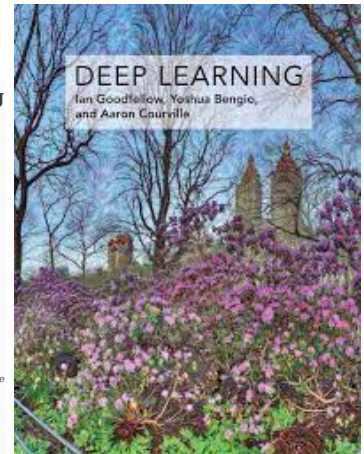
Deep Learning Books

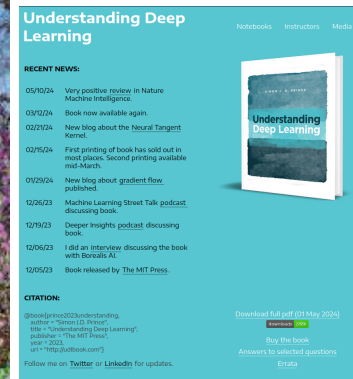
Probabilistic Machine Learning An Introduction

by Kevin Patrick Murphy.
MIT Press, March 2022.

Key links

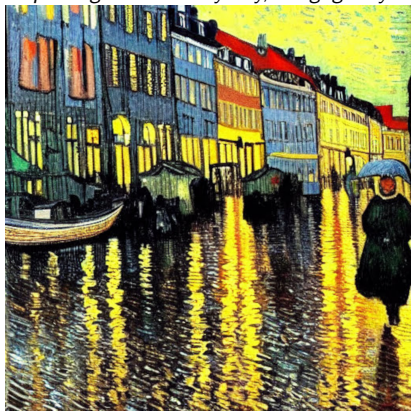
- Short table of contents
- Long table of contents
- Preface
- Draft pdf file, 2023-06-21. CC-BY-NC-ND license. (Please cite the official reference below.)
- Report issues here
- Order a hardcopy from MIT Press or Amazon..
- Figures from the book (png files)
- Code to reproduce most of the figures
- Diff from 2012 book
- Solutions to (non-starred) exercises





Big Data & Deep Learning Revolution

copenhagen on a rainy day, vangogh style



Stable Diffusion [1].

Trained on 500M images using 100's of GPU days

- Transformed multiple domains: computer vision, NLP
- Impactful inter-disciplinary applications
- Techniques from couple of decades ago [2,3]; matured in the past decade
- Accelerated by access to *Big Data* and *Big Compute*

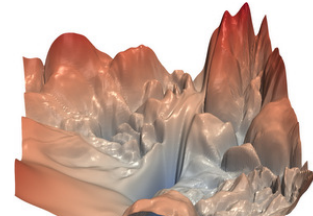
[1] Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models.
[2] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1985). Learning internal representations by error propagation.

[3] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition.

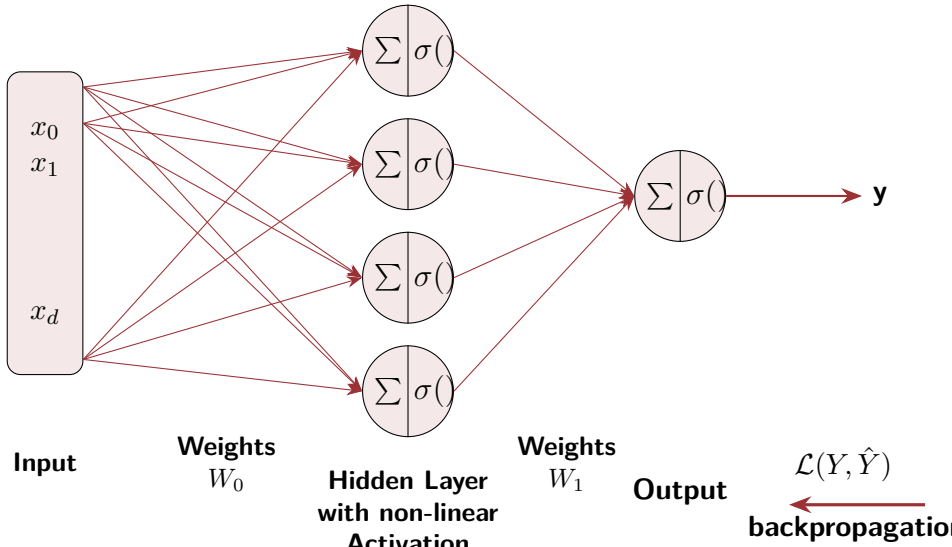
Connecting the dots so far

Observed data	$\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N\} : \mathbf{x}_i \in \mathbb{R}^{C \cdot H \cdot W \cdot D}$
Labels / Targets	$\mathbf{Y} = \{\mathbf{y}_1, \mathbf{y}_2, \ldots, \mathbf{y}_N\} : \mathbf{y}_i \in \mathbb{R}^{M \cdot H \cdot W \cdot D}$
Decision functions/ Models	$f_\theta(\cdot) : \mathbf{X} \rightarrow \mathbf{Y}$
Data distribution	$p(\mathbf{X})$
Joint distribution	$p(\mathbf{X}, \mathbf{Y})$
Modeled distribution	$p_\theta(\mathbf{X})$, parameterised by θ

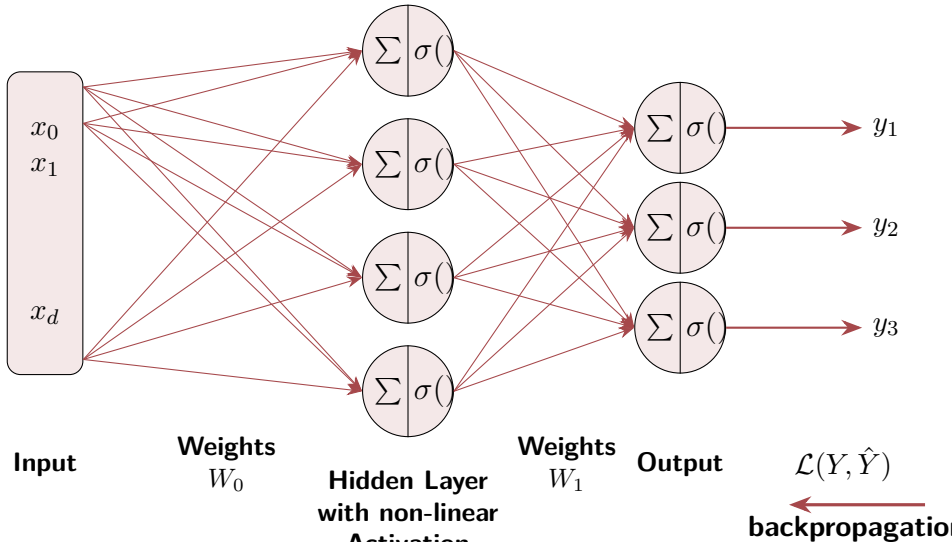
- $f_\theta(\cdot)$ can be any Deep Learning model (MLPs, CNNs, Transformers...)
- Over-parameterised, non-linear function approximator
- θ are the trainable parameters; $|\theta| >> N$
- Model selection from massive hypothesis sets
- Usually, requires large labelled datasets



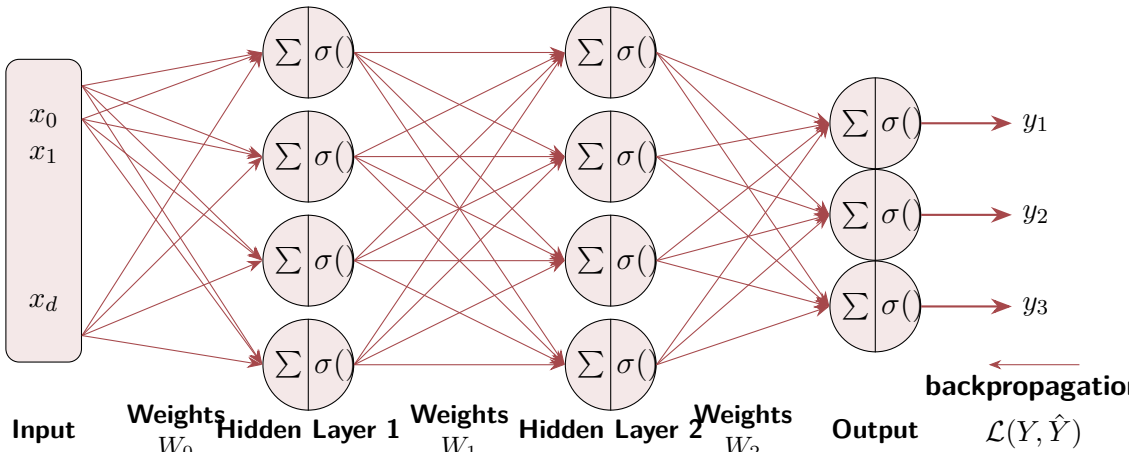
First Deep Learning Model: MLP



MLP with Multiple Outputs



MLP with Multiple Hidden Units and Multiple Outputs



Backpropagation Summary

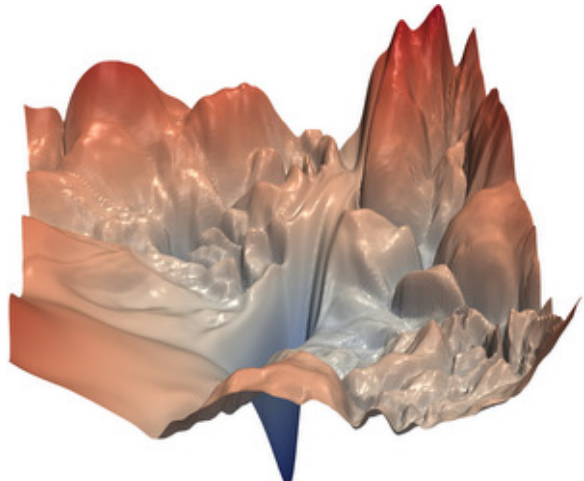
The 'Secret Sauce' of Deep Learning is just recursive calculus applied to massive matrices.
To optimize $\mathbf{w}$, we repeat these three phases until convergence:

1. **The Forward Pass (Signal Flow):** Input $\mathbf{x}$ flows through the layers. We compute pre-activations $z^{(\ell)} = \mathbf{w}^{(\ell)}\mathbf{a}^{(\ell-1)} + b^{(\ell)}$ and activations $\mathbf{a}^{(\ell)} = \sigma(z^{(\ell)})$ until we reach the output $\hat{y}$ and calculate the Loss $\mathcal{L}(y, \hat{y})$.
2. **The Backward Pass (Error Credit Assignment):** We use the **Chain Rule** in reverse. We compute the error signal $\delta^{(\ell)}$ for each layer. $\rightarrow$ *'How much did each neuron contribute to the final error?'*
3. **The Parameter Update (The Nudge):** We compute the gradient $\nabla_{\mathbf{w}}\mathcal{L}$. We then nudge every weight in the opposite direction:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \nabla_{\mathbf{w}}\mathcal{L}$$

Computational graph logic: Signal goes forward, 'blame' goes backward.

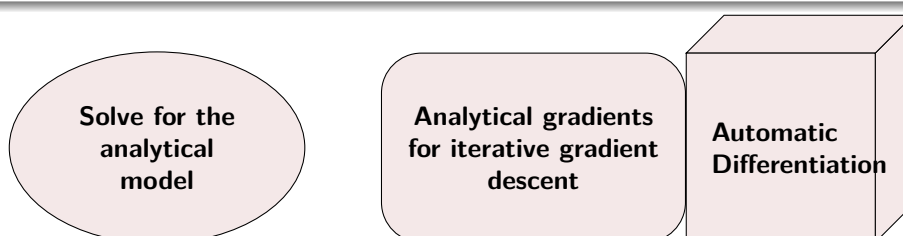
Increasing number of tunable parameters gives more flexibility but...



Gradient based optimization in different scenarios

Maximum likelihood estimation under-the-hood, always!

$\mathbf{w}^* = \arg\min_{\mathbf{w}} -log\ell(D; \mathbf{w})$



Linear Regression

Logistic Regression

Almost everything else.
Including DL

AutoDiff: The Chain Rule Foundation

To compute the gradient of the loss $\mathcal{L}$ with respect to a weight $w_{ij}^{(\ell)}$, we decompose the global derivative into local components:

The Local Derivative Chain

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(\ell)}} = \underbrace{\frac{\partial \mathcal{L}}{\partial a_i^{(\ell)}}}_{\text{Upstream Error}} \cdot \underbrace{\frac{\partial a_i^{(\ell)}}{\partial z_i^{(\ell)}}}_{\text{Local Sensitivity}} \cdot \underbrace{\frac{\partial z_i^{(\ell)}}{\partial w_{ij}^{(\ell)}}}_{\text{Local Input}}$$

- **Pre-activation sum:** $z_i^{(\ell)} = \sum_j w_{ij}^{(\ell)} a_j^{(\ell-1)} + b_i^{(\ell)}$
- **Sensitivity:** if $a_i^{(\ell)} = \sigma(z_i^{(\ell)})$, this is just $\sigma'(z_i)$
- **Dependency:** The gradient depends on the upstream error signal and the local input activation $a_j^{(\ell-1)}$.

AutoDiff: Defining the Error Signal δ

The algorithm simplifies the chain by combining the first two terms into a single vector $\delta^{(\ell)}$.

The Mapping

$$\text{Error Signal } \delta_i^{(\ell)} = \underbrace{\frac{\partial \mathcal{L}}{\partial a_i^{(\ell)}} \cdot \frac{\partial a_i^{(\ell)}}{\partial z_i^{(\ell)}}}_{\text{Upstream error \& Local Sensitivity}}$$

- **Vectorized Backprop Step (Recursive):**

$$\delta^{(\ell)} = \underbrace{\left((W^{(\ell+1)})^T \delta^{(\ell+1)} \right)}_{\text{Error from layer above}} \odot \underbrace{\sigma'(z^{(\ell)})}_{\text{Local sensitivity}}$$

- **Weight Update (The Final Product):**

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \delta^{(\ell)} \times \underbrace{(a^{(\ell-1)})^T}_{\text{Input signal}}$$

Why $(W^T\delta)$? The Multivariate Chain Rule

How does the scalar Upstream Error $\frac{\partial \mathcal{L}}{\partial a_i^{(\ell)}}$ become a matrix-vector product?

1. In a dense layer, $a_i^{(\ell)}$ influences **every** pre-activation $z_k^{(\ell+1)}$ in the next layer.
2. the multivariate chain rule:

$$\frac{\partial \mathcal{L}}{\partial a_i^{(\ell)}} = \sum_k \underbrace{\frac{\partial \mathcal{L}}{\partial z_k^{(\ell+1)}}}_{\delta_k^{(\ell+1)}} \cdot \underbrace{\frac{\partial z_k^{(\ell+1)}}{\partial a_i^{(\ell)}}}_{W_{ki}^{(\ell+1)}}$$

3. **Vectorization:** This summation $\sum_k \delta_k^{(\ell+1)} W_{ki}^{(\ell+1)}$ is the definition of the i-th element of the vector:

$$(\mathbf{W}^{(\ell+1)})^{\mathbf{T}} \delta^{(\ell+1)}$$

The Sigmoid and derivative

The sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$ has a unique property for its derivative:

- **Derivation:**

$$
\begin{aligned}
\sigma'(x) &= \frac{d}{dx}(1+e^{-x})^{-1} \\
&= -(1+e^{-x})^{-2} \cdot (-e^{-x}) \\
&= \frac{e^{-x}}{(1+e^{-x})^2} \\
&= \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}}
\end{aligned}
$$

The Sigmoid and derivative

The sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$ has a unique property for its derivative:

- **Derivation:**

$$\begin{aligned}\sigma'(x) &= \frac{d}{dx}(1+e^{-x})^{-1} \\ &= -(1+e^{-x})^{-2} \cdot (-e^{-x}) \\ &= \frac{e^{-x}}{(1+e^{-x})^2} \\ &= \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}}\end{aligned}$$

- **Final Form:**

$$\sigma'(x) = \sigma(x)(1-\sigma(x))$$

Why this matters for AutoDiff

If we know the output of the neuron, calculating the gradient requires zero exponential operations: just one subtraction and one multiplication!

Activity: Think pair share

System Parameters:

- Input Activation $a^{(0)} = 2.0$
- Target $y = 1.0$ (Positive)
- Current Weight $W^{(1)} = -1.0$
- Learning Rate $\eta = 0.1$
- Loss $\mathcal{L} = \frac{1}{2}(a^{(1)} - y)^2$

The Challenge

The model is currently 12% confident. Use the Backprop equations to find the new weight.

Vevox Poll

Update Direction of W?

A More Positive

B More Negative

C No Change

Solution 1: The Forward Pass

Symbolic Logic

Pre-activation:

$$z^{(1)} = W^{(1)} a^{(0)}$$

Activation:

$$a^{(1)} = \sigma(z^{(1)})$$

Numerical Result

$$z^{(1)} = -1.0 \times 2.0 = \mathbf{-2.0}$$

$$a^{(1)} = \sigma(-2.0) \approx \mathbf{0.12}$$

Solution 2: The Backward Pass (δ)

Symbolic Logic

Error Signal (δ):

$$\delta^{(1)} = \frac{\partial \mathcal{L}}{\partial a^{(1)}} \cdot \sigma'(z^{(1)})$$

Weight Gradient:

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \delta^{(1)} (a^{(0)})^T$$

Numerical Result

$$\delta^{(1)} = \underbrace{(0.12 - 1)}_{\text{Upstream}} \cdot \underbrace{(0.12 \cdot 0.88)}_{\text{Sensitivity}}$$

$$\delta^{(1)} \approx \mathbf{-0.093}$$

$$\text{Grad} = -0.093 \times \underbrace{2.0}_{\text{Input}}$$

$$\nabla_W \mathcal{L} \approx \mathbf{-0.186}$$

Solution 3: Optimization Update

Symbolic Logic
Update Rule:

$$W^{(1)} \leftarrow W^{(1)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(1)}}$$

Numerical Result

$$W^{(1)} = -1.0 - (0.1 \times -0.186)$$

$$W^{(1)} = -1.0 + 0.0186$$

$$W^{(1)} \approx \mathbf{-0.981}$$

The weight became more positive.

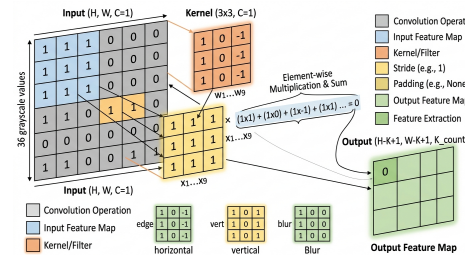
Overview

- Introduction
- The Ingredients of ANN
- Optimization algorithm - Gradient Descent
- Optimization objective function
- Deep Learning
- Architectures

CNN: Spatial Inductive Bias

Key Properties:

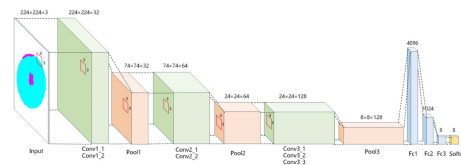
- **Local Connectivity:** Neurons only look at a small patch (receptive field) of the input.
- **Shared Weights:** Convolutional kernels (filters) are slid across the entire input.
- **Feature Maps:** Captures spatial hierarchies (edges → textures → objects).
- **Efficiency:** Drastically reduces the number of parameters compared to fully connected MLPs.



CNN: Pooling and Invariance

Advanced Features:

- **Pooling Layers:** Downsamples the feature maps (e.g., Max Pooling) to reduce dimensionality.
- **Translation Invariance:** The model can recognize a pattern regardless of where it appears in the frame.
- **Deep Hierarchies:** Modern CNNs (ResNet, EfficientNet) stack hundreds of layers for complex vision tasks.
- **Beyond Images:** Also used for 1D signals (ECG) and graph-structured data.



RNN: Sequential Dependencies

Key Properties:

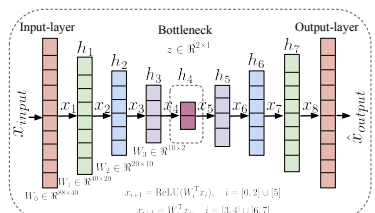
- **Temporal Processing:** Designed for data where order matters (time-series, text, audio).
- **Recurrence:** Hidden state h_t is computed using the current input x_t and the previous state h_{t-1}.
- **Parameter Sharing:** The same weights are used at every time step, allowing for variable-length sequences.
- **Memory:** Theoretically capable of retaining information from the distant past.

Autoencoder: The Latent Bottleneck

Das, Sneha, et al. "Towards transferable speech emotion representation: on loss functions for cross-lingual latent representations." ICASSP. IEEE, 2022.

Key Properties:

- **Unsupervised Learning:** Learns efficient data codings by training the network to reconstruct its own input.
- **The Bottleneck:** A hidden layer with a smaller dimension than the input, forcing the model to learn salient features.
- **Non-linear PCA:** Acts as a deep learning extension to the dimensionality reduction methods seen in Week 8.
- **Reconstruction Loss:** Objective is to minimize $L(x, \hat{x})$, typically using Mean Squared Error (MSE).

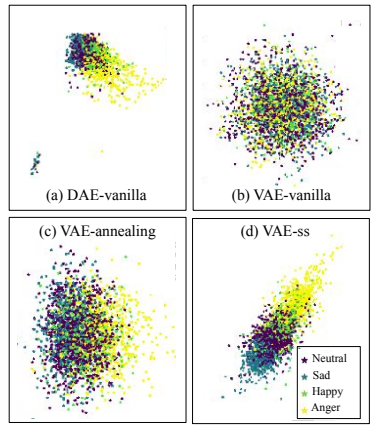


Autoencoder: Applications & Variants

Das, Sneha, et al. "Towards transferable speech emotion representation: on loss functions for cross-lingual latent representations." ICASSP. IEEE, 2022.

Advanced Features:

- **Denoising AEs:** Learns robust features by reconstructing clean data from partially corrupted inputs.
- **Variational AEs (VAEs):** Imposes a probabilistic structure on the latent space, enabling data generation.
- **Feature Extraction:** The encoder can be used as a pre-trained backbone for downstream supervised tasks.

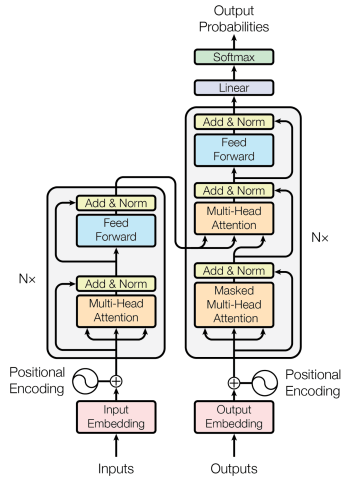


Transformer: The Attention Mechanism

Vaswani, Ashish, et al. "Attention is all you need." Advances in neural information processing systems 30 (2017).

Key Properties:

- **Self-Attention:** Allows every token to weigh the importance of every other token in the sequence simultaneously.
- **Global Context:** Captures long-range dependencies better than RNNs/LSTMs.
- **No Recursion:** Computes all positions in parallel, significantly speeding up training on GPUs.
- **Inductive Bias:** Minimal assumptions about data structure compared to CNNs.

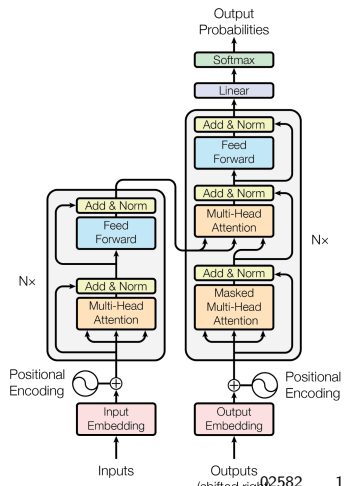


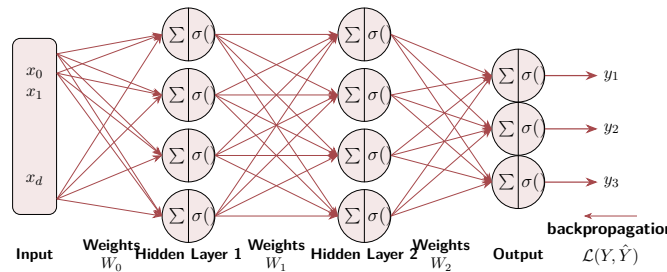
Transformer: Scalability & Multi-Head Attention

Vaswani, Ashish, et al. "Attention is all you need." Advances in neural information processing systems 30 (2017).

Advanced Features:

- **Multi-Head Attention:** Allows the model to attend to information from different representation subspaces at different positions.
- **Positional Encodings:** Since there is no recursion, inject 'order' information directly into the input embeddings.
- **Scalability:** Foundation for Large Language Models (LLMs) like GPT and BERT.
- **Residual Connections:** Uses 'Add & Norm' blocks to allow training of very deep stacks.



Summary

- Deep Learning models use many parameters to approximate non-linear functions
- MLPs consist of logistic regression-type neurons
- MLPs can approximate arbitrarily complex functions
- Can be used for diverse array of tasks
- Deeper or Wider MLPs are better?
- No obvious ways to decide architectures
- Automatic differentiation is the secret sauce that makes deep learning possible